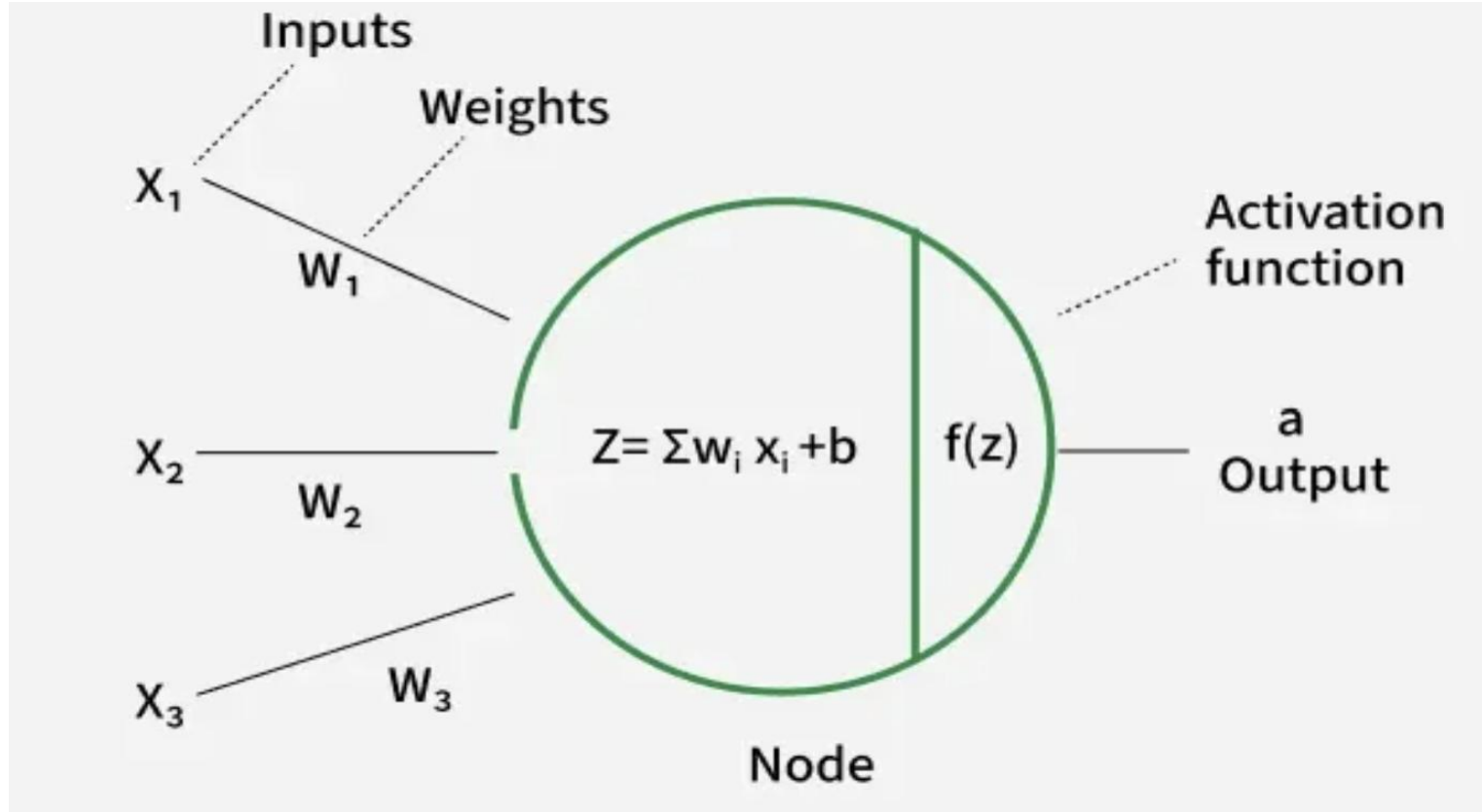


Activation Function in NN

Activation Function:

- An activation function in deep learning is a mathematical equation that determines the output of a neural network's node, or “neuron.”
- It decides whether a neuron should be “activated” or not based on the weighted sum of inputs and a bias term, i.e., whether the information that the neuron holds is important enough to pass to the next layer.
- It introduces non-linearity, enabling the model to learn and represent complex data patterns.

Activation Function:



Activation Function: Example

Inputs: $i1, i2$

Hidden layer: neurons $h1$ and $h2$

Output layer: one neuron (output)

Weights: $w1, w2, w3, w4, w5, w6$

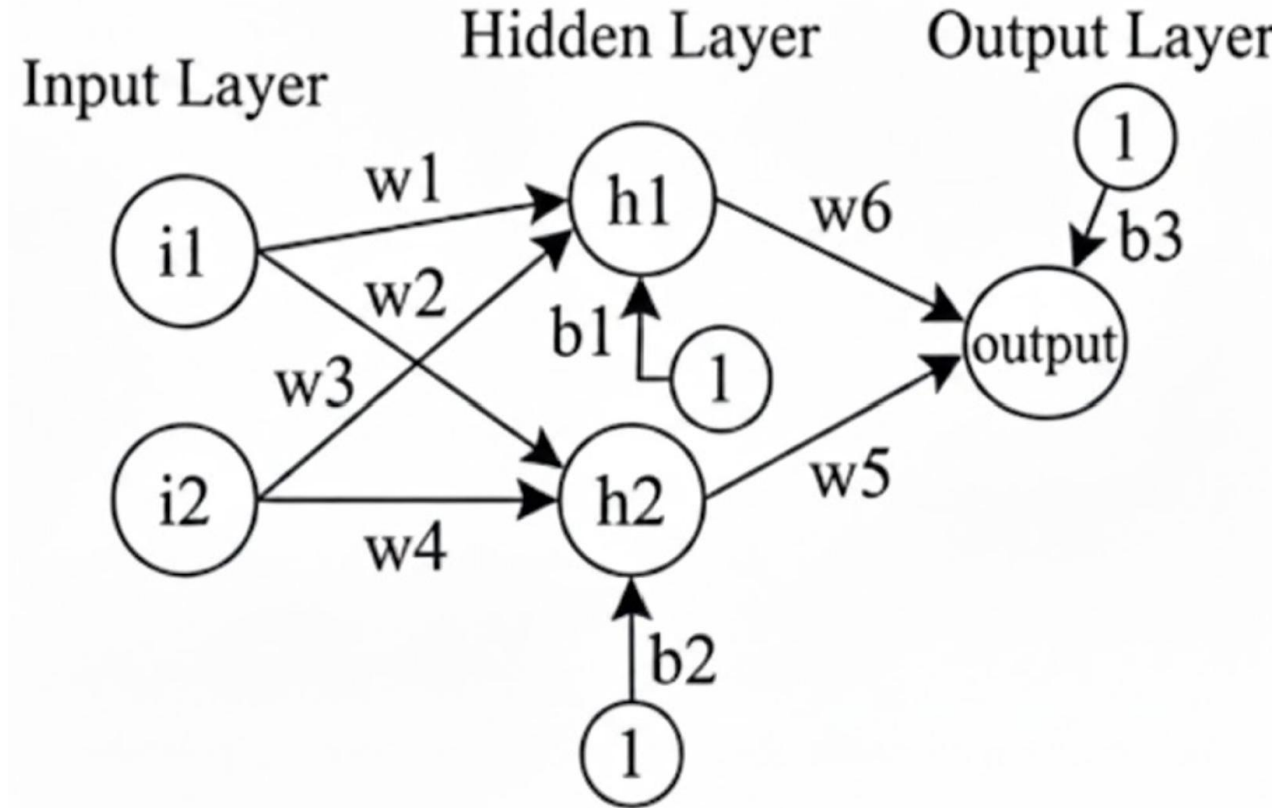
Biases: $b1, b2$ for hidden layer,
 $b3$ for output layer

$$h1 = i1 * w1 + i2 * w3 + b1$$

$$h2 = i1 * w2 + i2 * w4 + b2$$

The output before activation is:

$$\text{Out} = h1 * w6 + h2 * w5$$



Without activation, these are linear equations.

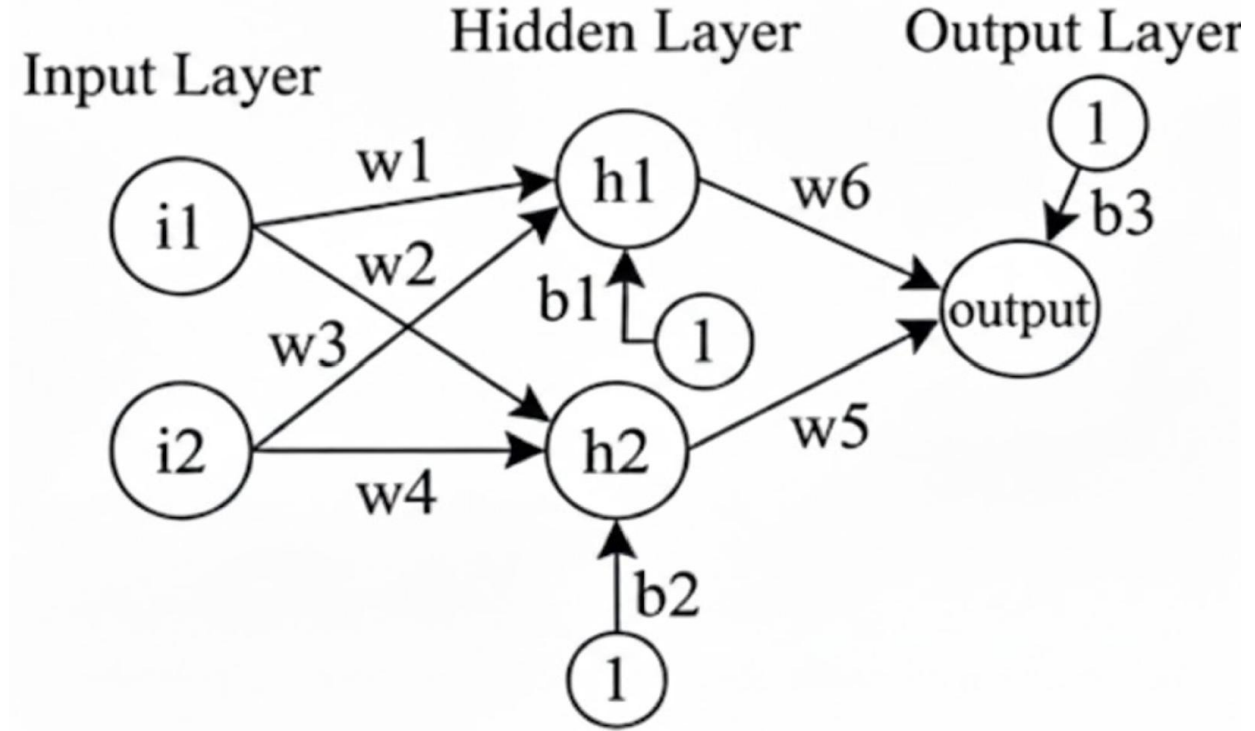
Activation Function: Example

$$h1 = i1 \cdot w1 + i2 \cdot w3 + b1$$

$$h2 = i1 \cdot w2 + i2 \cdot w4 + b2$$

The output before activation is:

$$\text{Out} = h1 \cdot w6 + h2 \cdot w5 + b3$$



Without activation, these are linear equations.

To introduce non-linearity, we apply a sigmoid activation:

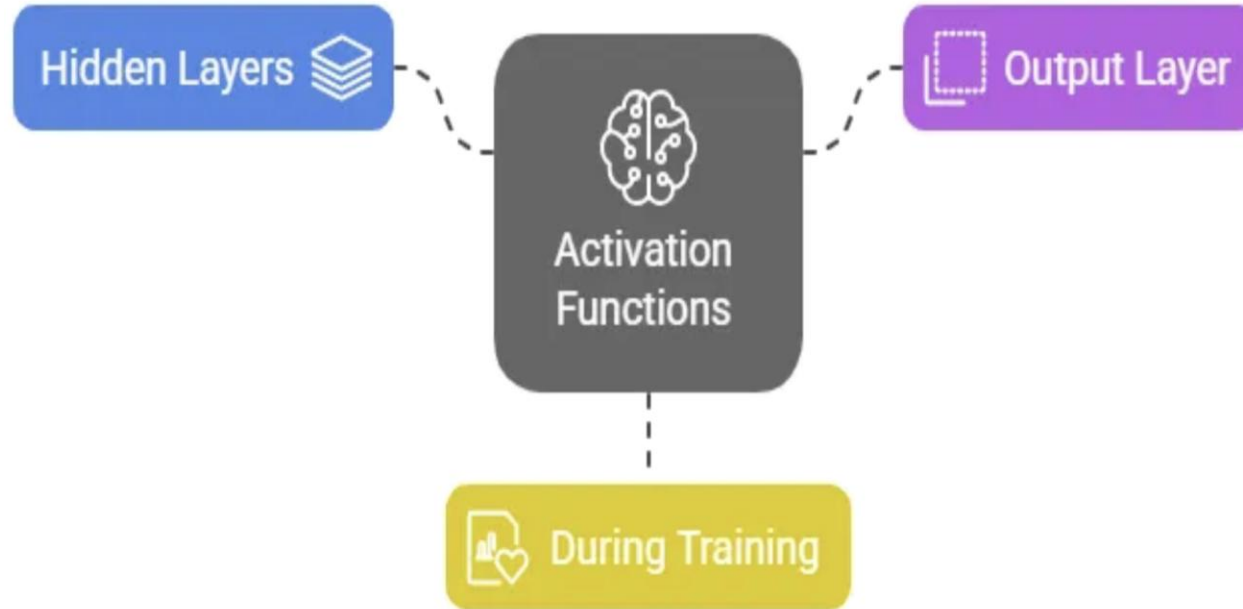
$$\sigma(x) = \frac{1}{1+e^{-x}}$$

$$\text{Out} = \sigma(h1 \cdot w6 + h2 \cdot w5 + b3)$$

Why Non-Linearity is Important?

- Most real-world data is non-linear.
- Activation functions allow the network to learn non-linear relationships between inputs and outputs.
- Non-linear functions allow neural networks to form curved decision boundaries, making them capable of handling complex patterns (e.g., classifying apples vs. bananas under varying colors and shapes).
 - Linear models create only straight-line boundaries in classification

When Are Activation Functions Used?



When Are Activation Functions Used?

Hidden Layers: Activation functions are applied after each layer's computation to introduce non-linearity and increase the model's complexity.

Output Layer: The activation function depends on the type of task:

- Sigmoid for binary classification
- Softmax for multi-class classification
- Linear for regression

When Are Activation Functions Used?

- The Forward Pass: In the forward pass of a neural network, the activation function is applied after the weighted sum of inputs is computed in each neuron. (During Training)
- The Backward Pass (Backpropagation):
 - During backpropagation, the model computes the gradient of the loss with respect to weights and biases. This requires differentiating the activation function.
 - If the activation function wasn't there (or was purely linear), the gradient would be the same everywhere, and the network wouldn't be able to "fine-tune" its understanding of complex data. (During Training)

Types of Activation Functions:

Different functions behave differently, and choosing the wrong one can lead to poor model performance.

Linear Activation Function (Identity Function):

It is defined as

$$f(x) = x$$

- It simply returns the input x as the output.
- Output Range: $-\infty$ to $+\infty$
- Use Case: Only used in regression problems, where we want the model to predict continuous values.
- Example: Predicting house prices or temperature.
- Can't learn complex patterns
- Doesn't introduce non-linearity
- Stacking layers doesn't improve learning

Sigmoid Activation Function

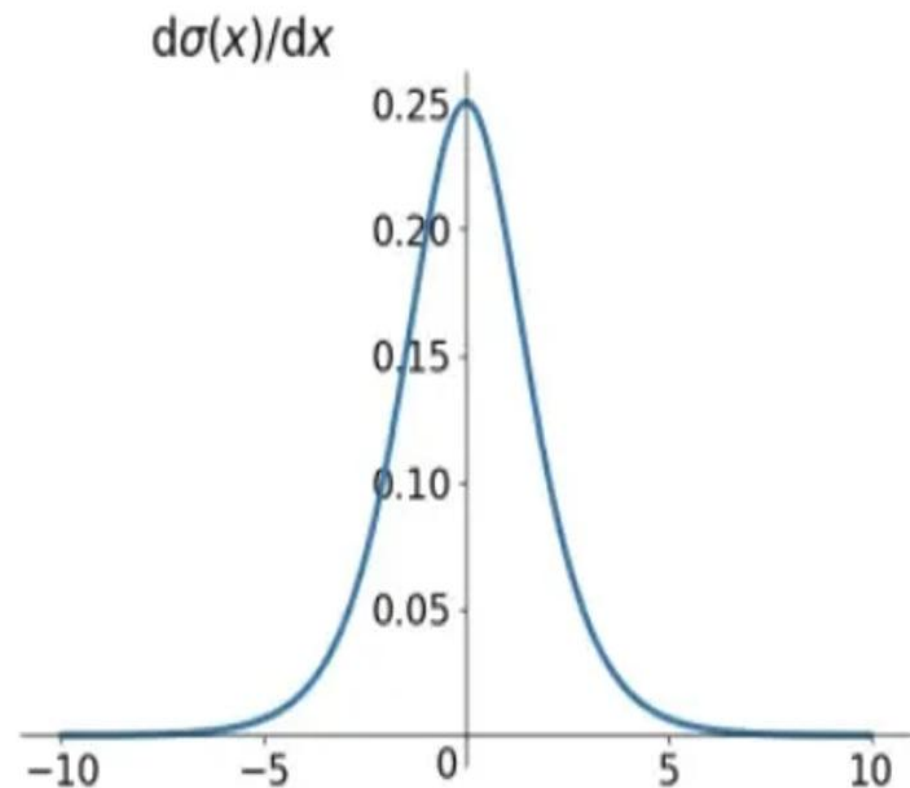
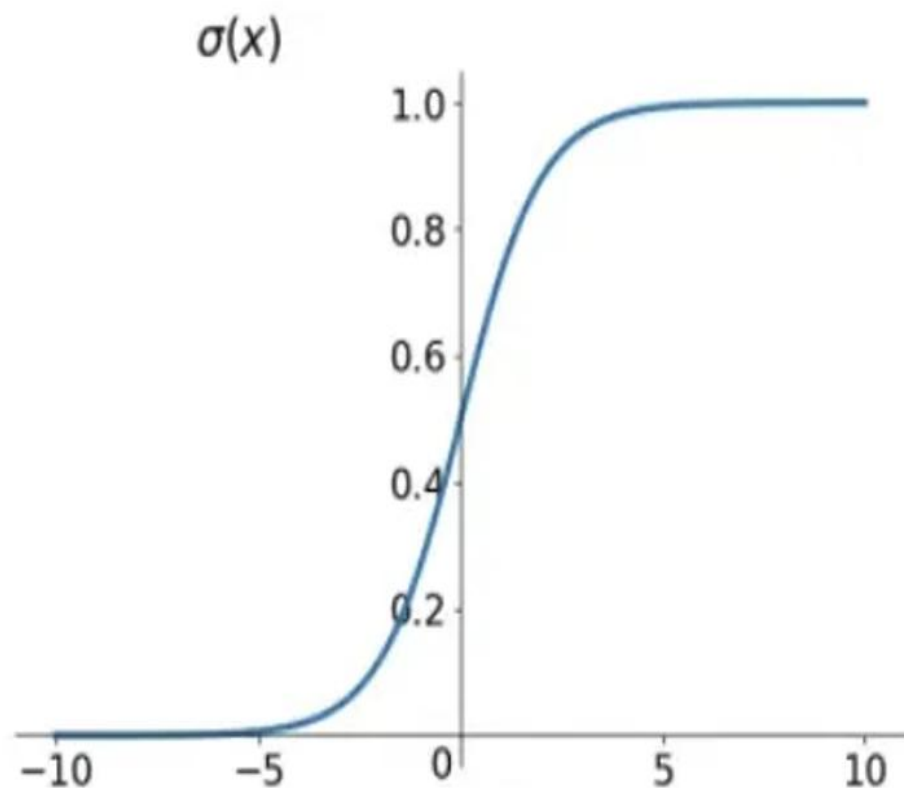
It is defined as

$$f(x) = \frac{1}{1 + e^{-x}}$$

- Output Range: 0 to 1
- Use Case: Binary classification problems — when we want to predict probabilities.
- Example: Predicting whether an email is spam or not.
- Maps any value between 0 and 1
- Useful for probabilistic interpretation
- Vanishing Gradient Problem: Very small derivatives for large values of x
- Not zero-centered

Sigmoid Activation Function

$$f(x) = \frac{1}{1 + e^{-x}}$$



Sigmoid function and its derivative

Tanh (Hyperbolic Tangent)

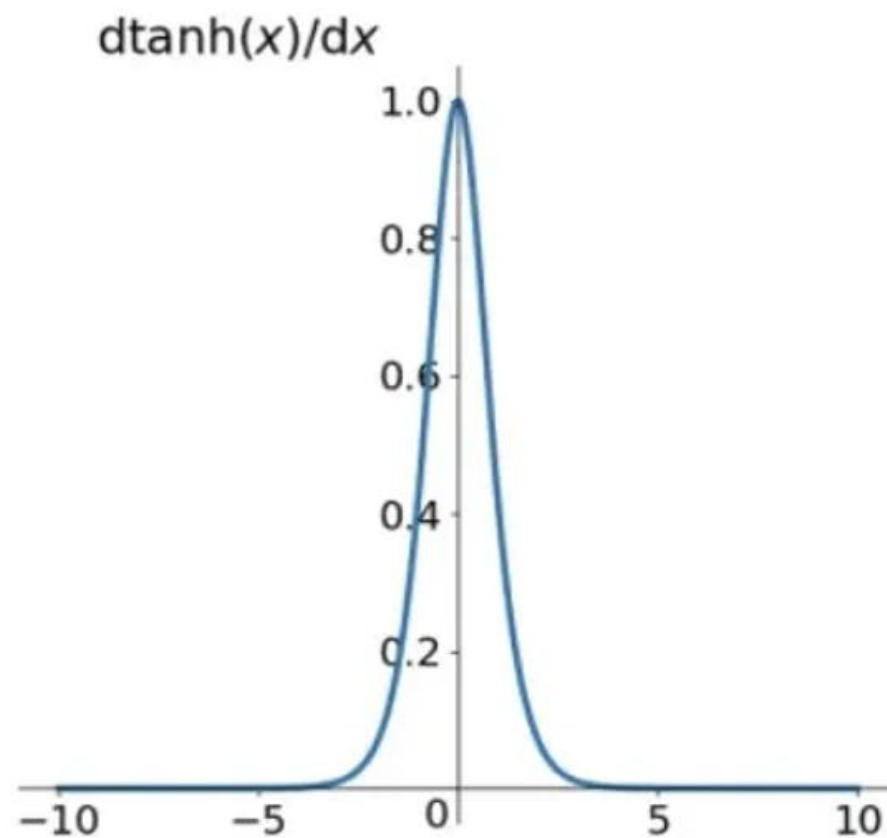
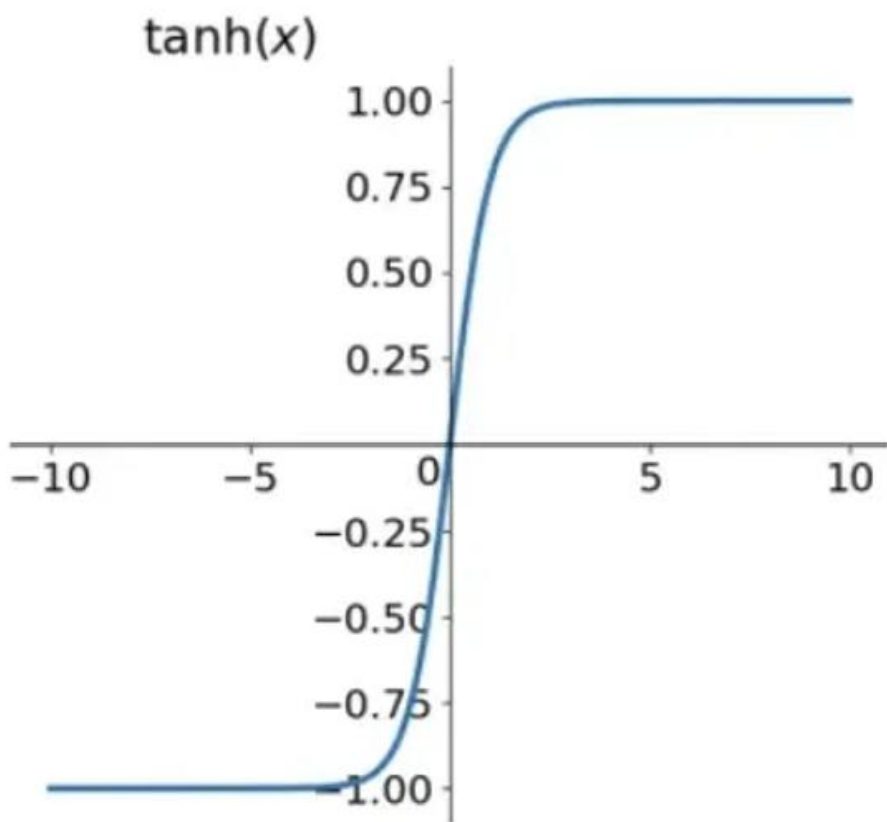
It is defined as

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- Output Range: -1 to 1
- Use Case: Preferred over sigmoid in hidden layers when data is centered.
- Example: NLP models like RNNs used Tanh in hidden layers.
- Zero-centered
- Stronger gradients than sigmoid
- Still suffers from vanishing gradient problem

Tanh (Hyperbolic Tangent)

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



The plot of tanh and its derivative

Softmax Function

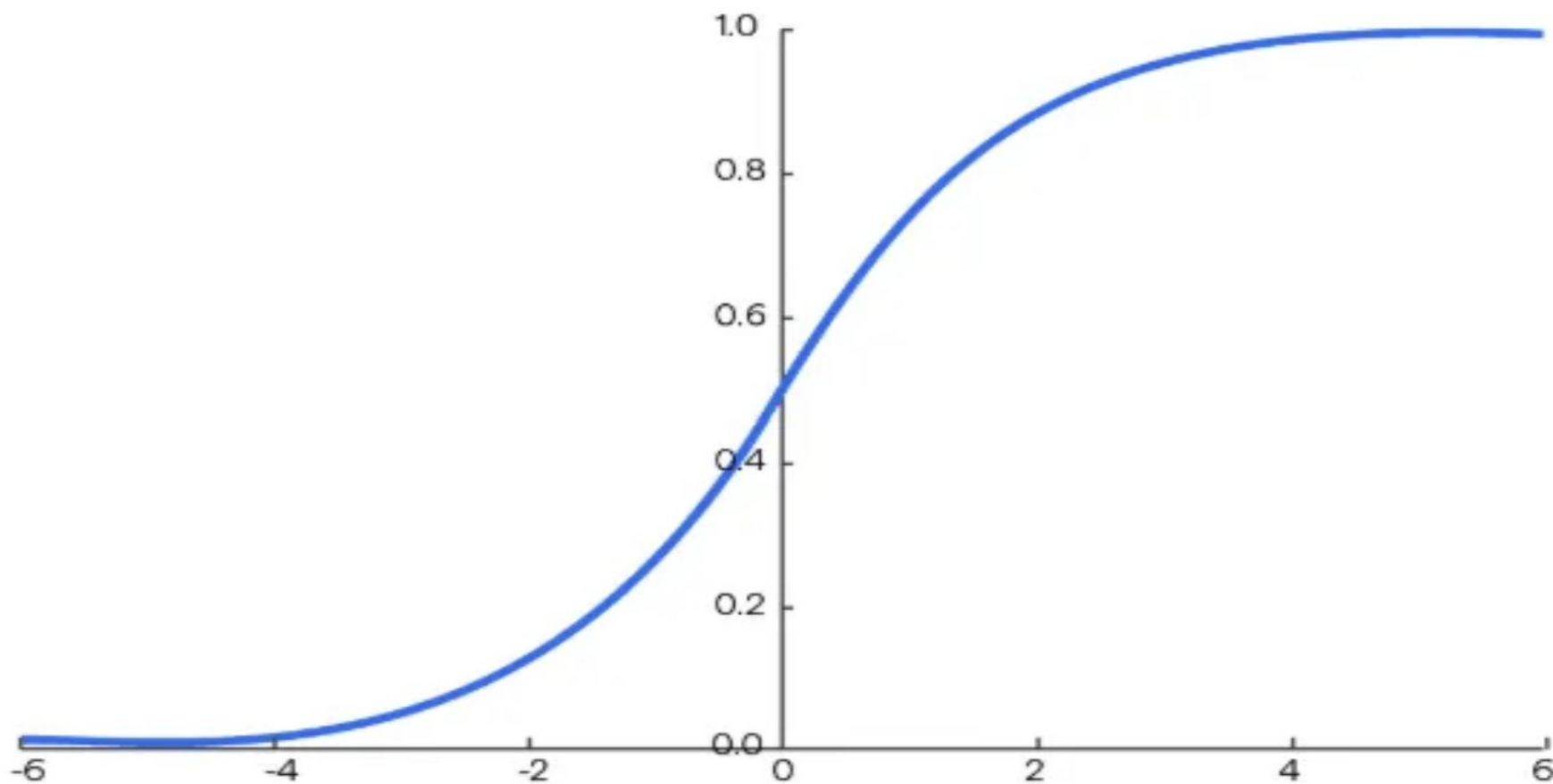
For input vector x with elements $x_1, x_2, \dots, x_C$, the softmax function is defined as:

$$f(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$

- Output Range: Each output is between 0 to 1 (values sum up to 1)
- Use Case: Multi-class classification problems
- Example: Digit recognition (0–9), emotion detection, language translation
- Gives probabilities across multiple classes
- Suitable for classification tasks
- Sensitive to outliers and large values

Softmax Function

$$f(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$



Softmax activation function

ReLU (Rectified Linear Unit)

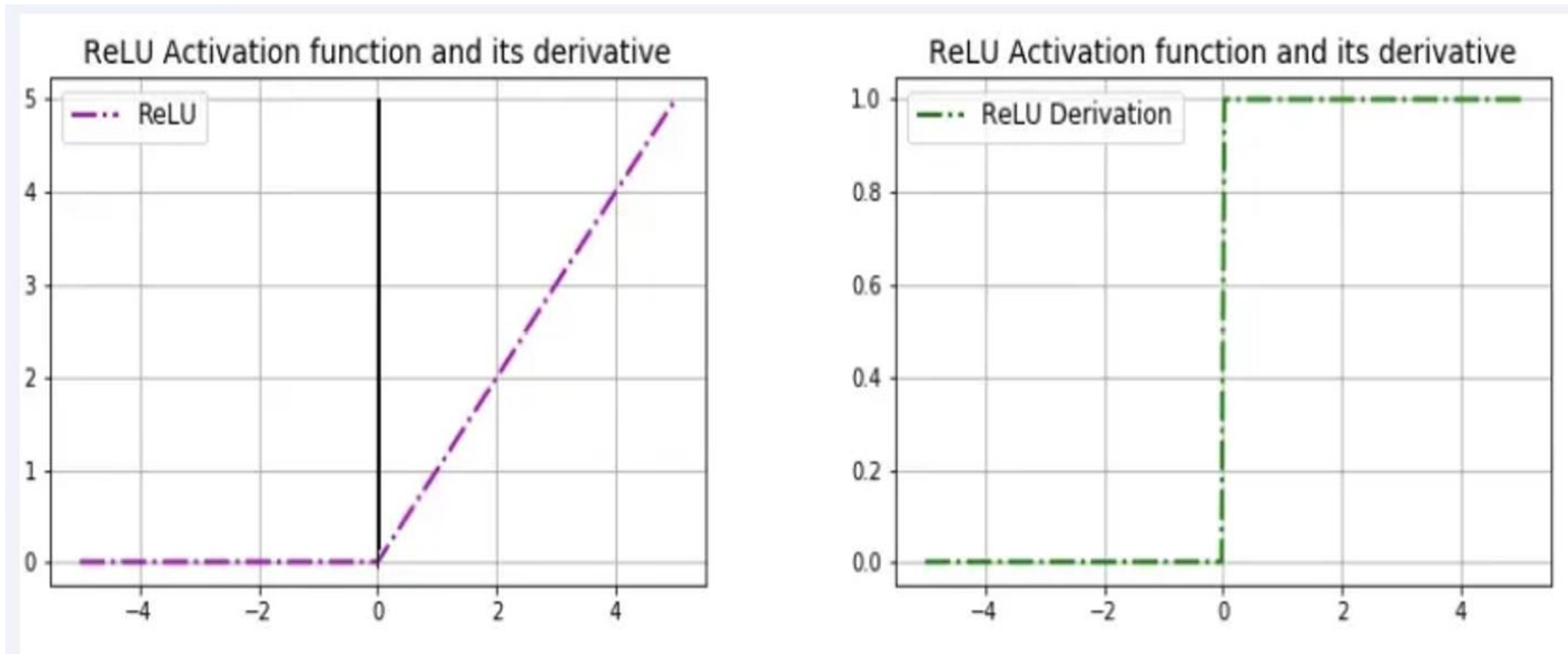
It is defined as

$$f(x) = \max(0, x)$$

- Output Range: 0 to ∞
- Use Case: Default activation function in most deep neural networks.
- Example: Image classification, object detection (CNNs), language models (Transformers)
- Computationally efficient
- Helps prevent vanishing gradients
- Sparse activation — only some neurons fire
- Dying ReLU problem — neurons can become inactive (always output 0)

ReLU (Rectified Linear Unit)

$$f(x) = \max(0, x)$$



Leaky ReLU

It is defined as

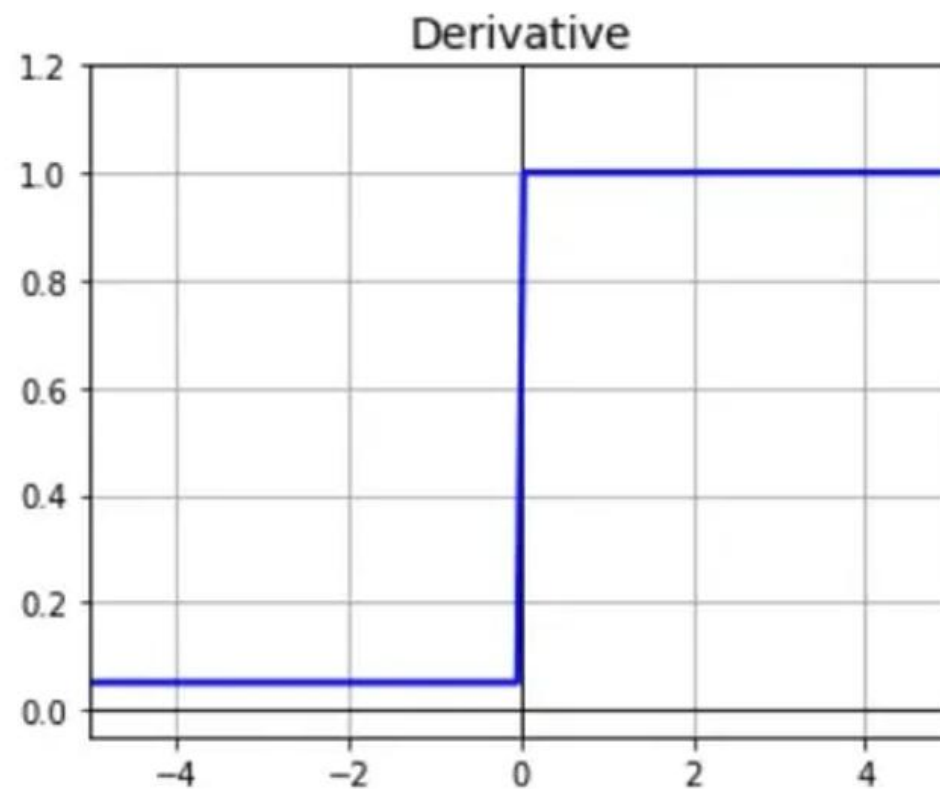
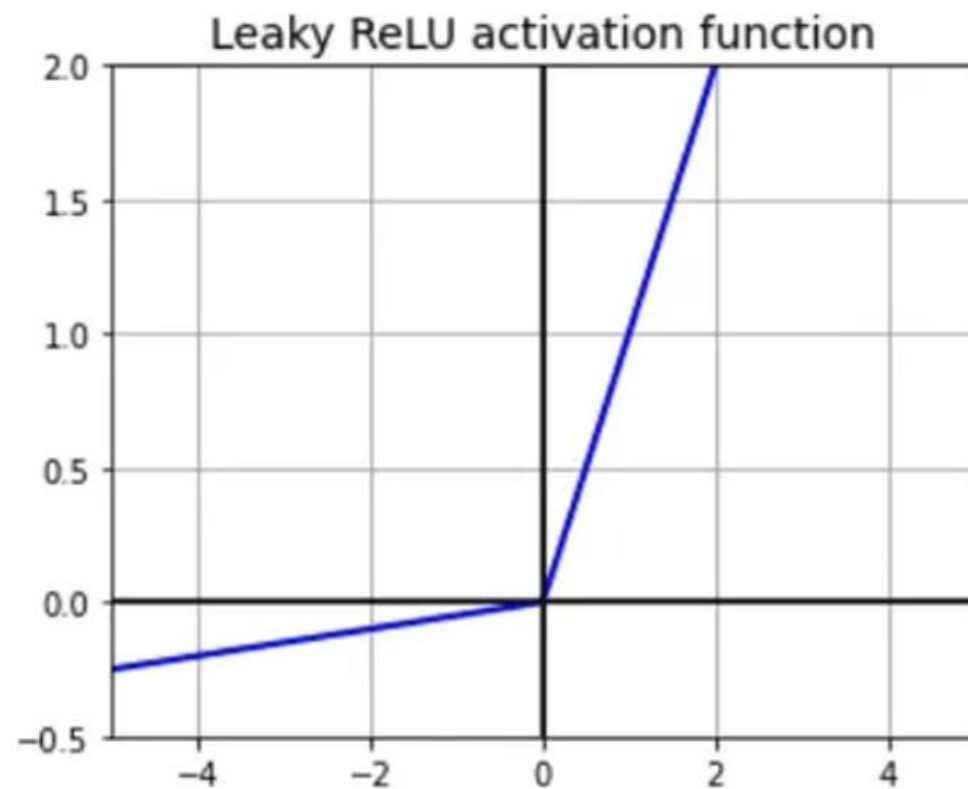
$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases}$$

where α is a small value (e.g., 0.01)

- Use Case: To fix dying ReLU issue.
- Example: Used in GANs and deeper CNNs
- Allows small gradient when $x < 0$
- Solves dying ReLU problem
- Introduces a small slope for negative inputs, which may or may not help depending on the dataset and learning dynamics

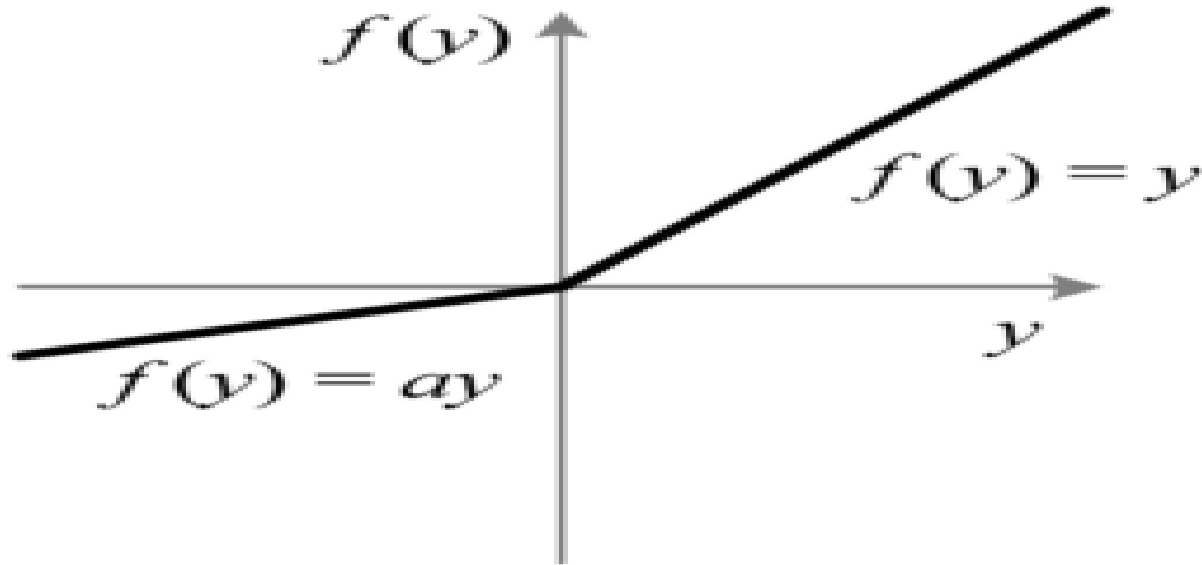
Leaky ReLU

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases}$$



Parametric ReLU (PReLU)

$$f(y_i) = \begin{cases} y_i, & \text{if } y_i > 0 \\ a_i y_i, & \text{if } y_i \leq 0 \end{cases}$$



- Parametric ReLU (PReLU) is an advanced variation of the traditional ReLU and Leaky ReLU activation functions.
- Output Range: $-\infty$ to $+\infty$

Parametric ReLU (PReLU)

- PReLU uses $\mathbf{a}_i$ as a hyperparameter that the model learns during the training process - Advantage
- PReLU offers an increase in the accuracy of the model.
- Fine-tuning this parameter ($\mathbf{a}_i$) can be time-consuming, especially with diverse datasets - Disadvantage
- To fix dying ReLU issue
- The PReLU retains the non-zero gradient for negative inputs, similar to the Leaky ReLU. This helps mitigate the vanishing gradient problem, promoting more stable and effective training in deep neural networks

Parametric ReLU (PReLU)

$$f(y_i) = \begin{cases} y_i, & \text{if } y_i > 0 \\ a_i y_i, & \text{if } y_i \leq 0 \end{cases}$$

- If $a_i=0$, the activation function f becomes ReLU.
- If $a_i>0$ and fixed, the activation function f becomes Leaky ReLU.
- If a_i is a learnable parameter, the activation function f becomes PReLU.

Example: Very deep CNNs with large datasets (image recognition, object detection)

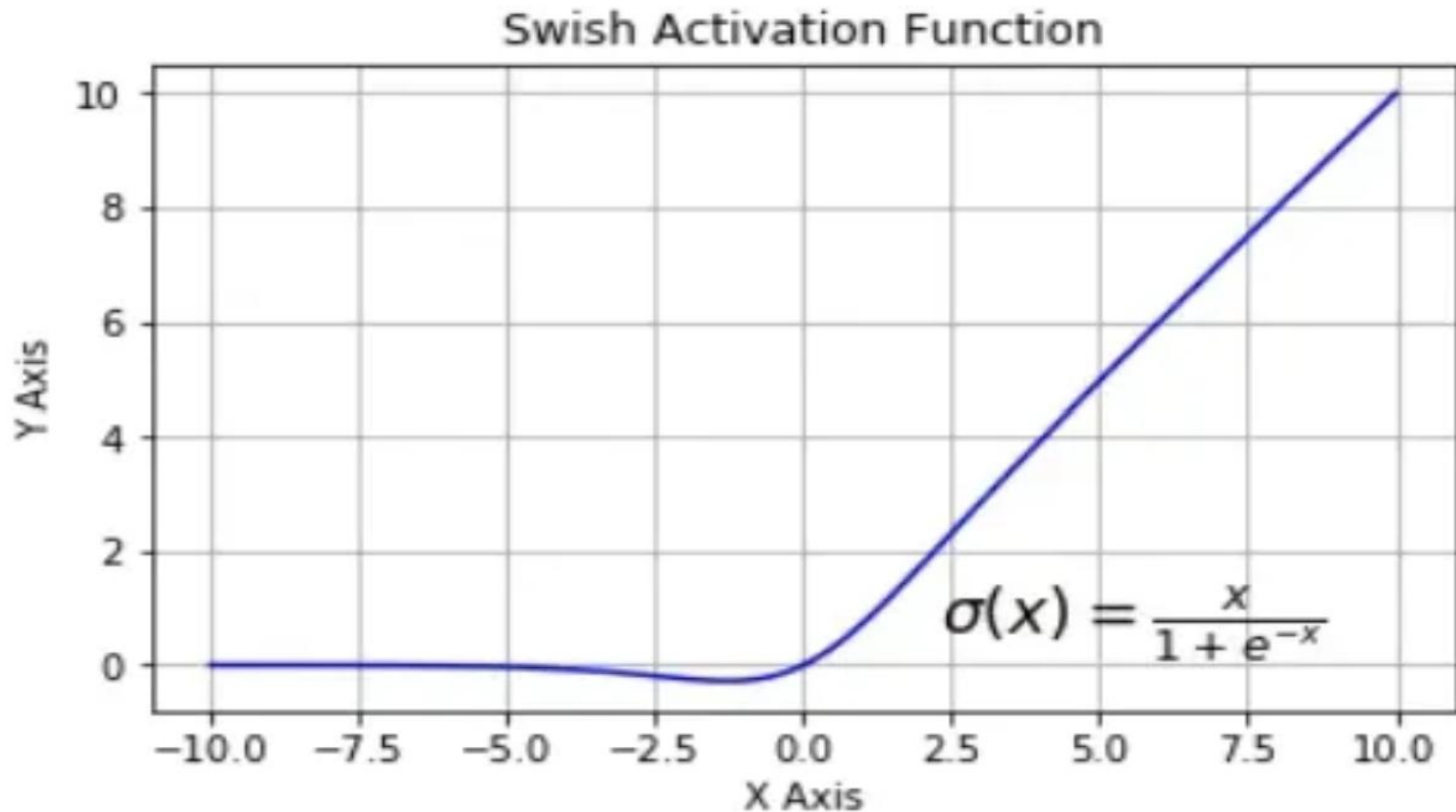
Swish (Developed by Google)

It is defined as: $f(x) = x \cdot \text{sigmoid}(x)$

- Output Range: Varies (non-monotonic)
- Use Case: Used in cutting-edge architectures like EfficientNet
- Example: Deep learning for image classification or medical imaging
- Smooth and non-linear
- Outperforms ReLU in some deep models
- Slightly more computationally expensive than ReLU

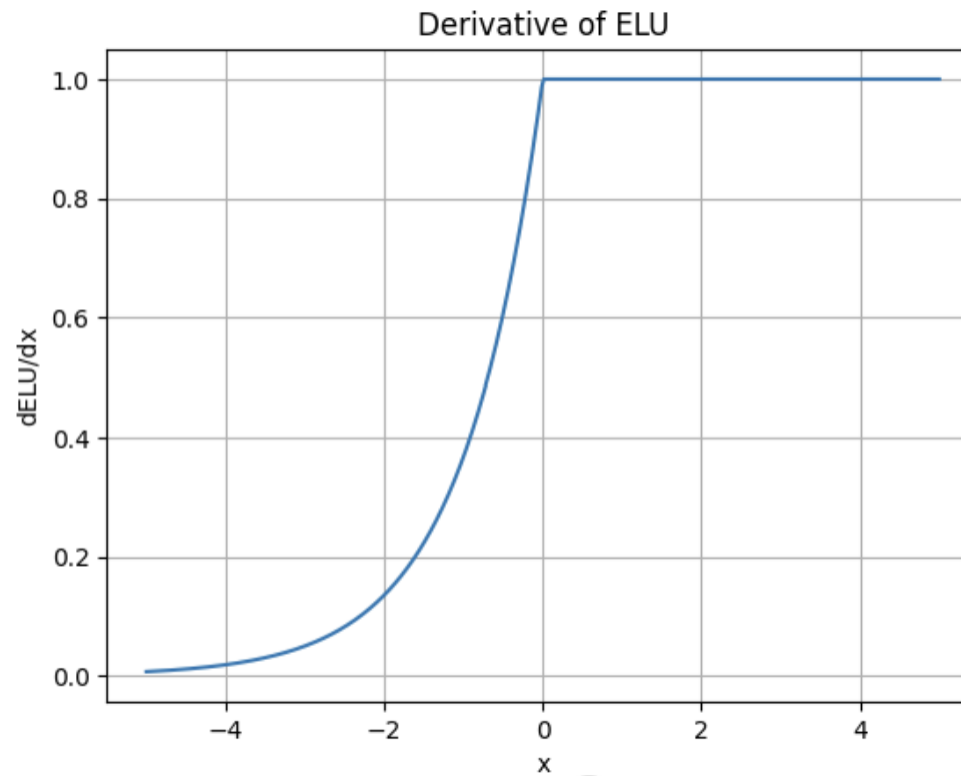
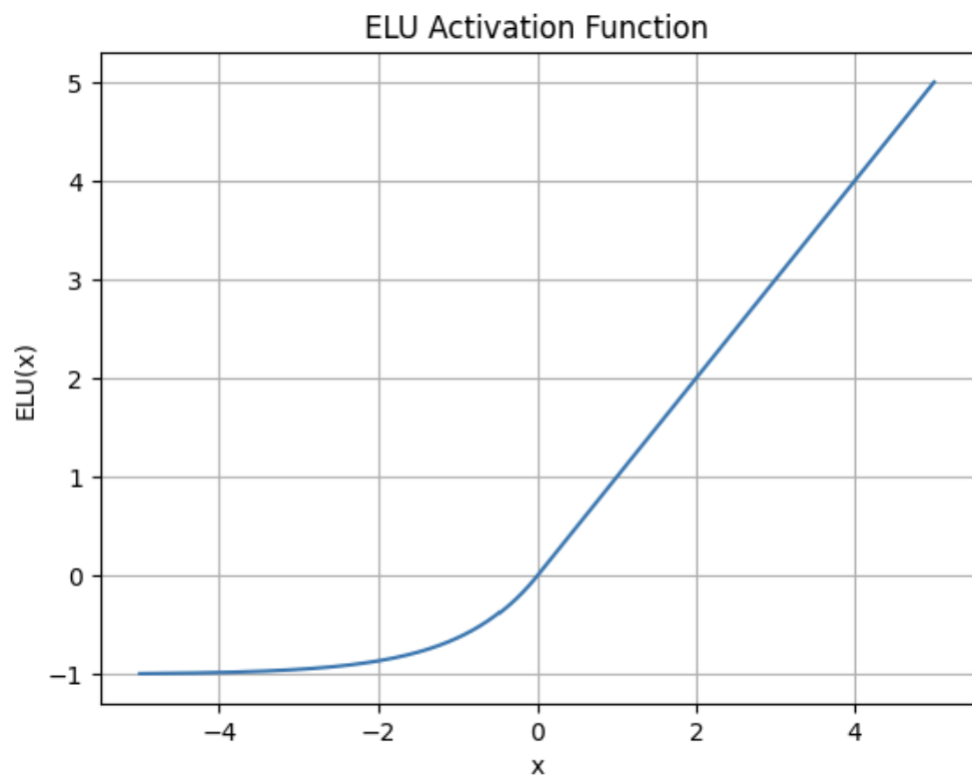
Swish (Developed by Google)

$$f(x) = x \cdot \text{sigmoid}(x)$$



Exponential Linear Units (ELU)

$$f(x) = \begin{cases} x, & \text{if } x > 0 \\ \alpha(e^x - 1), & \text{if } x \leq 0 \end{cases}$$



Exponential Linear Units (ELU) Activation Function

- Exponential Linear Unit (ELU) modifies the negative part of ReLU by applying an exponential curve. It allows small negative values instead of zero which improves learning dynamics.
- Does not have a vanishing gradients problem
- Does not have an exploding gradients problem
- Does not have a dying ReLU issue
- ELU is slower to compute, but ELU compensates this by faster convergence during training
 - During test time ELU is slower to compute than ReLU.

Exponential Linear Units (ELU)

- Output Range: $-\alpha$ to $+\infty$
- Example: Used in general deep neural networks
 - Autoencoders
 - Use ELU if training is unstable or slow to converge with ReLU

The Gaussian Error Linear Unit (GELU)

- GELU is derived from a smooth approximation of the cumulative distribution function (CDF) of the standard normal distribution. It is the input times the standard normal CDF at that point

$$GELU(x) = x \times CDF(x) = x \times \frac{1}{2} \left(1 + erf \left(\frac{x}{\sqrt{2}} \right) \right)$$

- In its original formulation, GELU can't be used in the networks as the function is complex and slow to compute. Therefore, we can reformulate it as:

$$GELU_{\tanh}(x) = 0.5x \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right) \right)$$

The Gaussian Error Linear Unit (GELU)

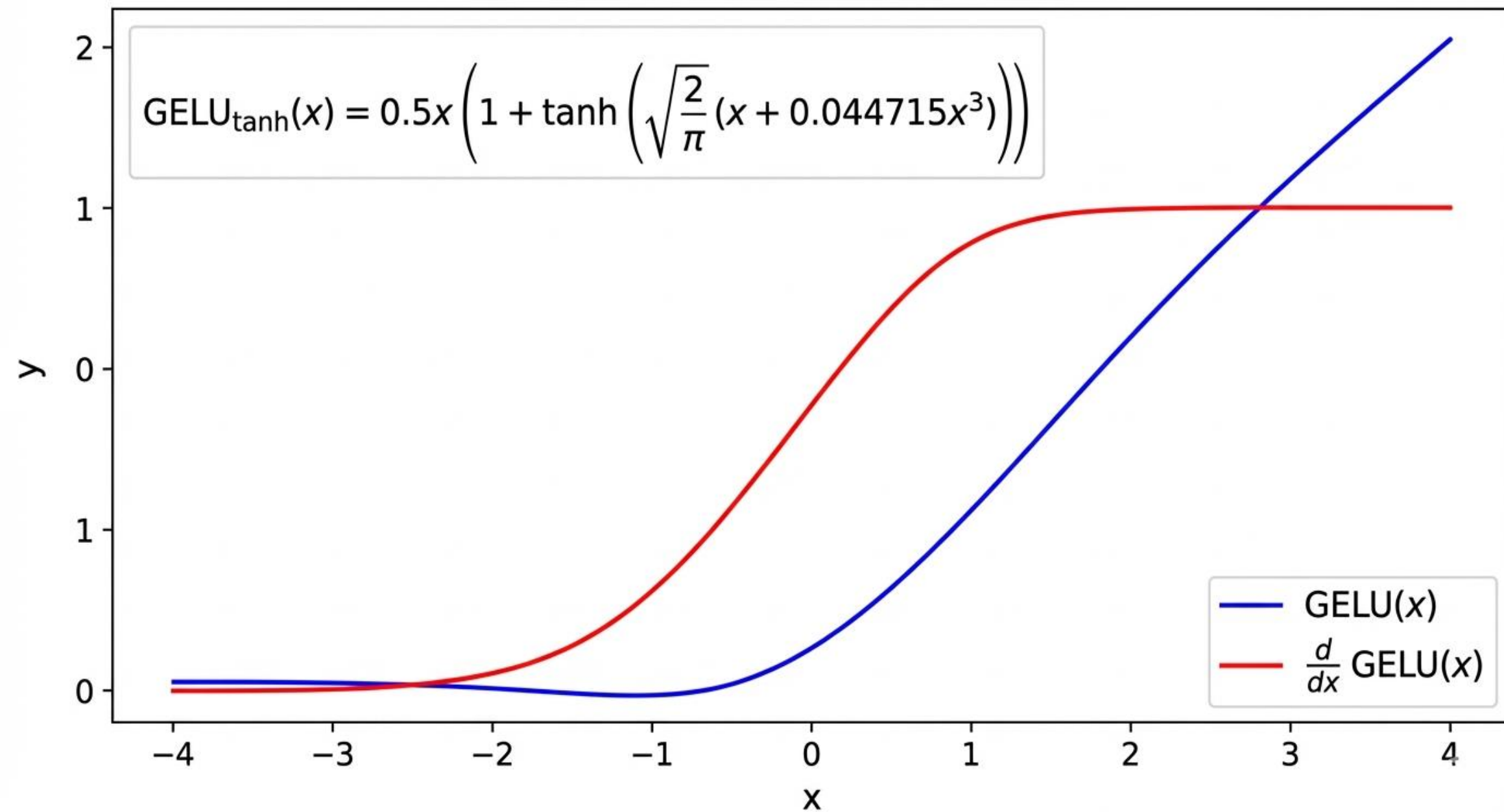
- We approximate the Gaussian error function using the tanh function
- The scaling factor $\sqrt{2/\pi}$ applies normalization, ensuring the output is within a reasonable range
- Simpler sigmoid-based approximation

$$GELU_{\text{sigmoid}}(x) = x \times \text{sigmoid}(1.702 \times x)$$

- The benefits of the tanh and sigmoid approximations are that they and their derivatives are easy to compute.

GELU Function and its Derivative

$$\text{GELU}_{\tanh}(x) = 0.5x \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right) \right)$$



The Gaussian Error Linear Unit (GELU)

- Why Gelu

GELU Pros	GELU Cons
Addresses the vanishing gradient with no dead neurons	Increased computational complexity
Smoothness across the full range of input values	Performance can be task-specific
Non-monotonic behavior can capture complex patterns	Reduced interpretability due to increased complexity

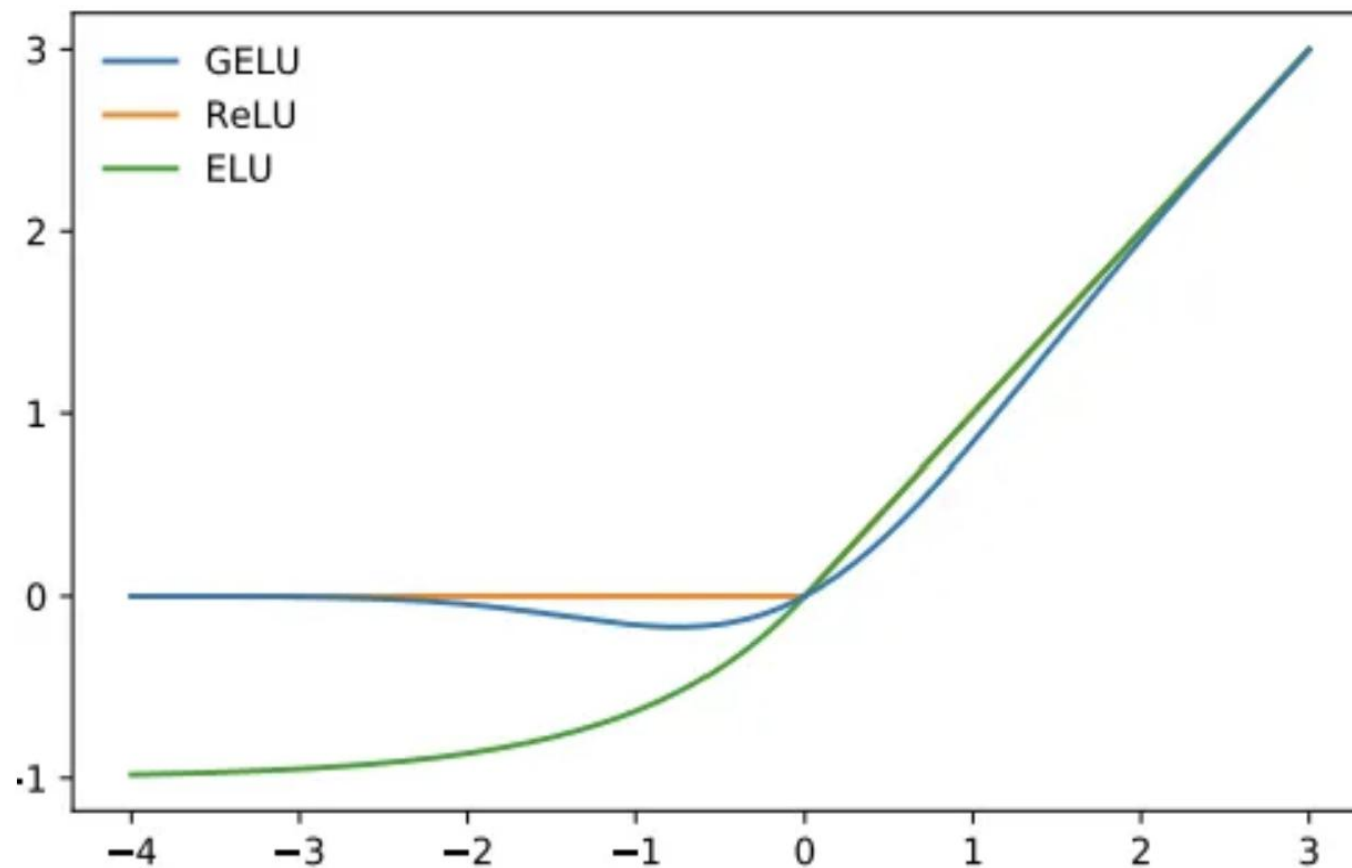
- GELU provides a smooth, non-monotonic, and differentiable alternative to traditional activation functions.

The Gaussian Error Linear Unit (GELU):

Example:

- Large Language Models (LLMs):
 - In the GPT series and other generative text models
 - NLP tasks like text summarization or sentiment analysis
- Vision Transformers (ViT):
 - Models that adapt the Transformer architecture for image classification rely heavily on GELU
 - High accuracy on benchmarks like ImageNet.

GELU, ReLU, ELU



GELU ($\mu=0, \sigma=1$), ReLU and ELU ($\alpha=1$)

SwiGLU (Swish-Gated Linear Unit)

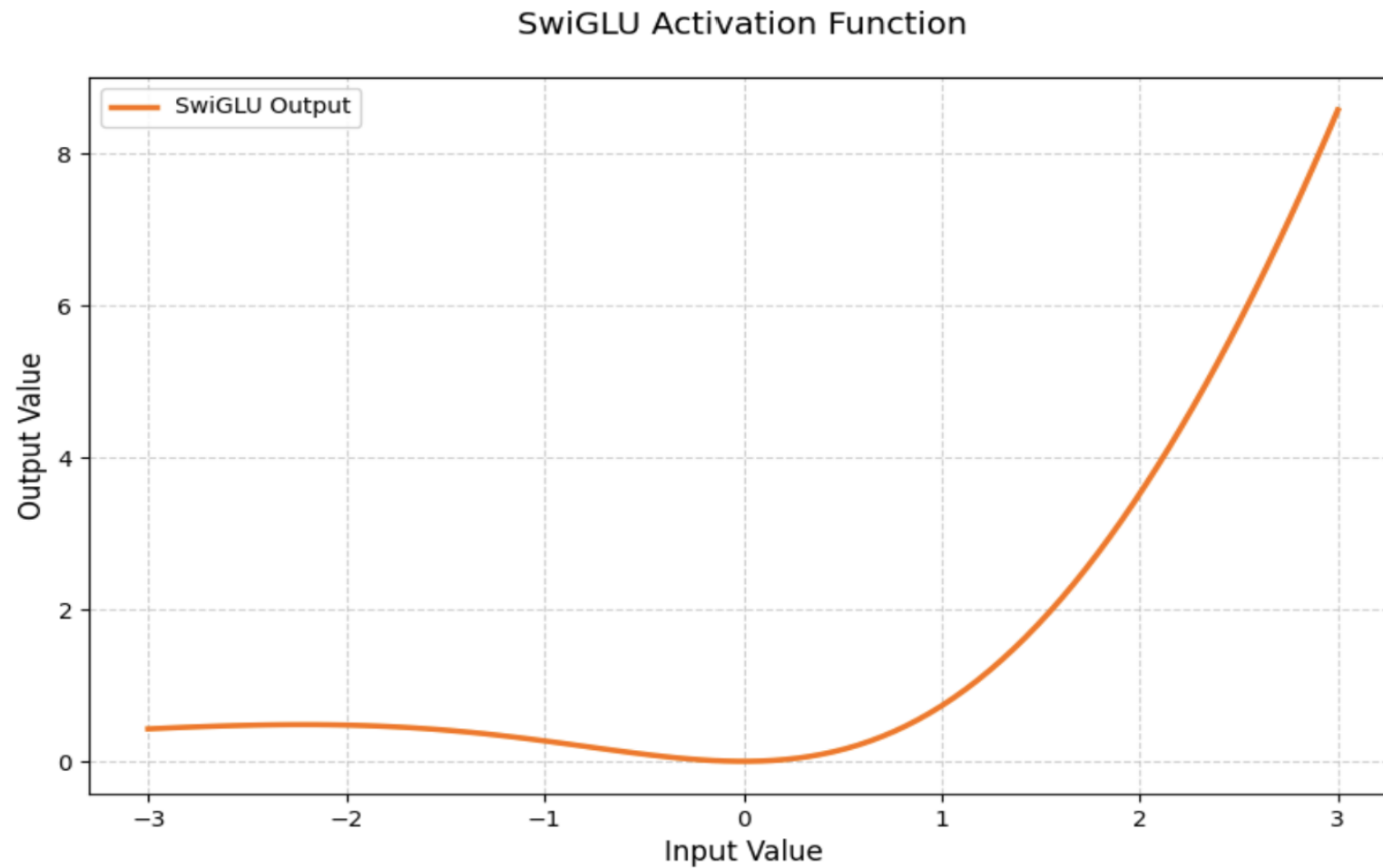
- SwiGLU is a variant of the Gated Linear Unit (GLU)
- SwiGLU is a gated activation function.
- Gated activation functions are designed to selectively pass information through the network, which can help in better capturing complex patterns in the data

$$\text{SwiGLU}(x) = (xW_1 + b_1) \odot \text{Swish}(xW_2 + b_2)$$

Where,

- $\text{Swish}(x) = x \cdot \sigma(x)$
- σ is the sigmoid function. It's a smoother function than ReLU that doesn't completely kill negative values, which helps gradients flow during training
- $\odot$ is an element-wise multiplication.

SwiGLU (Swish-Gated Linear Unit)



SwiGLU (Swish-Gated Linear Unit): Key properties

- Non-zero gradients for negative inputs
- It's a smooth, non-monotonic activation function that blends linear and non-linear behaviors
- Efficiency and Speed (with Hardware Acceleration)

Note:

- The latest Large Language Models (LLMs) like LLaMA and Google's PaLM are adopting a newer activation function: SwiGLU
- It can be used in Transformer-based models

Why SwiGLU is a Preferred Choice for LLMs

- The smooth, non-monotonic nature of the Swish function, combined with the gating mechanism, contributes to more stable training.
- This stability is important for training very deep LLMs, where gradient flow can be fragile.
- SwiGLU also helps prevent issues like vanishing or exploding gradients (unstable weight update).

Activation Function Comparison

Activation	Range	Non-Linear	Use Case	Gradient Risk
Linear	$-\infty$ to $+\infty$	✗ No	Regression output	No
Sigmoid	0 to 1	✓ Yes	Binary classification	Yes (Vanishing)
Tanh	-1 to 1	✓ Yes	NLP, Hidden layers	Yes (Vanishing)
ReLU	0 to $+\infty$	✓ Yes	Image, Deep networks	Yes (Dying ReLU)
Leaky ReLU	$-\infty$ to ∞	✓ Yes	Deeper CNNs, GANs	No
Softmax	0 to 1	✓ Yes	Multi-class classification	No
Swish	varies	✓ Yes	Efficient architectures	Low

Impact of Activation Functions on Model Performance

The choice of activation function has a direct impact on the performance of a neural network in several ways:

- **Convergence Speed:** Functions like ReLU allow faster training by avoiding the vanishing gradient problem while Sigmoid and Tanh can slow down convergence in deep networks.
- **Gradient Flow:** Activation functions like ReLU ensure better gradient flow, helping deeper layers learn effectively. In contrast Sigmoid can lead to small gradients, hindering learning in deep layers.
- **Model Complexity:** Activation functions like Softmax allow the model to handle complex multi-class problems, whereas simpler functions like ReLU or Leaky ReLU are used for basic layers.

Thank You